Project: To-Do List API **CR ID:** CR-002 **Title:** API Idempotency for Task Creation **Type:** Enhancement — Reliability **Priority:** High **Status:** Proposed

---

1. Problem Statement

`POST /api/tasks` has no protection against duplicate submissions. If a client sends a creation request and the response is lost (network timeout, dropped connection, client crash before the response is received), the client cannot safely retry — retrying blindly may create a second, duplicate task, since the server has no way to recognize the retry as "the same request" it already handled.

2. Proposed Change

Allow clients to attach an idempotency key to task-creation requests. If the same key is retried with the same request body, the API returns the original result without creating a second task. If the same key is reused with a *different* body, the API rejects the request, since that indicates a client bug rather than a legitimate retry.

This change is scoped to task creation only; no other endpoint's behavior changes.

3. Scenario

1. A client sends `POST /api/tasks` with an `Idempotency-Key` header and a task payload.
2. The server creates the task successfully and prepares a `201 Created` response — but the response is lost in transit due to a network failure.
3. The client, having received no response, retries the exact same request: same `Idempotency-Key`, same payload.
4. The server recognizes the key, recognizes the payload matches the original request, and returns the same `201 Created` response (with the same task id) as the first attempt — **without** creating a second task.
5. Later, a different client mistakenly reuses the same `Idempotency-Key` but with a different payload (a client bug). The server rejects this with `409 Conflict`, since reusing a key for a different request is invalid.

4. Functional Requirements

- **FR-01:** `POST /api/tasks` shall accept an optional `Idempotency-Key` HTTP header.
- **FR-02:** When a request includes an `Idempotency-Key` that has not been seen before, the server shall process it normally and record the key, a fingerprint of the request payload, and the resulting response.
- **FR-03:** When a request repeats a known `Idempotency-Key` with a payload matching the original request's fingerprint, the server shall return the original stored response **without** creating a new task.
- **FR-04:** When a request reuses a known `Idempotency-Key` with a payload that does **not** match the original fingerprint, the server shall reject the request with `409 Conflict`.
- **FR-05:** When a request omits the `Idempotency-Key` header, the server shall process it as an ordinary, non-idempotent creation (unchanged current behavior) — this header is opt-in, not mandatory, so existing clients are not broken.
- **FR-06:** Two simultaneous requests using the same new `Idempotency-Key` shall result in exactly one task being created; the second (losing) concurrent request shall receive the first request's result rather than creating its own task or erroring unpredictably.
- **FR-07:** Idempotency records shall not be retained indefinitely; a configurable expiration/cleanup policy shall be documented.

5. Acceptance Criteria

- Retrying an identical task-creation request with the same key produces only one task in the database, and both the original and retried calls return the same task id and data.
- Reusing a key with a different payload is rejected with `409 Conflict` and creates no task.
- Omitting the key behaves exactly as `POST /api/tasks` does today.
- Firing two identical requests with the same new key concurrently results in exactly one row in the `TASK` table.
- All existing task-creation test scenarios continue to pass unmodified when no `Idempotency-Key` is supplied.

6. Scope

In Scope:

- Idempotency for `POST /api/tasks` only.
- Persisted idempotency records (survive a server restart).
- Key reuse with a mismatched payload rejected.

Out of Scope:

- Idempotency for `PUT` / `DELETE` on tasks, or any Category endpoint, in this iteration (a natural future extension, not part of this CR).
- Client-side retry logic or SDKs — this CR covers only server behavior.
- Distributed idempotency-key storage across multiple application instances beyond what the shared Oracle database already provides (no separate cache layer such as Redis is introduced).

7. Approval

- **Business Approval:** Pending
- **Technical Approval:** Pending
- **Implementation Status:** Not Started